Cryptography and Security

Lecture 9: Message Authentication Codes (MACs)

Durham University

Department of Computer Science

Learning Objectives

By the end of this lecture, you will be able to:

1. Define message integrity and authentication requirements
2. Explain MAC security models and forgeability games
3. Construct MACs from pseudorandom functions (PRFs)
4. Analyze attacks on naive MAC constructions
5. Understand CBC-MAC construction and its security properties
6. Identify timing attacks and replay attack vulnerabilities

Lecture Overview

Today we will cover:

1. Message Integrity - Beyond confidentiality to authenticity
2. MAC Definitions - Formal security models and games
3. MAC Construction - Building MACs from PRFs
4. Security Analysis - Attacks and defenses
5. CBC-MAC - Efficient MAC from block ciphers

Message Integrity

Beyond Confidentiality

Confidentiality alone is insufficient:

- CPA-secure encryption protects against eavesdropping
- Active adversaries can modify or inject messages
- Recipients need to verify message authenticity

Real-world threats:

- Message modification attacks
- Message injection attacks
- Impersonation attacks
- Man-in-the-middle attacks

Authentication goals:

- Integrity: Message hasn’t been modified
- Authenticity: Message comes from claimed sender
- Non-repudiation: Sender cannot deny sending message

Critical insight: Encryption alone provides no integrity protection

Message Authentication Codes (MACs)

Basic MAC protocol:

1. Key sharing: Alice and Bob share secret key k
2. Authentication: Alice computes tag $t \leftarrow S_{k(m)}$ for message m
3. Transmission: Alice sends (m, t) to Bob
4. Verification: Bob checks if t is valid for m using key k

Key properties:

- Symmetric key primitive (shared secret required)
- Provides integrity and authenticity (but not confidentiality)
- Efficient computation and verification
- Unforgeable without knowing secret key

Usage: Often combined with encryption for authenticated encryption

Formal MAC Definition

Definition (Message Authentication Code)

A MAC consists of two probabilistic polynomial-time algorithms $\mathcal{I} = (S, V)$:

- S (Sign): Takes key k and message $m \in \{0,1\}^*$, outputs tag $t \leftarrow S_{k(m)}$
- V (Verify): Takes (k, m, t), outputs 1 (accept) or 0 (reject)

Correctness requirement: For all keys k and messages m: $V_k\left(m, S_{k(m)}\right) = 1$

Typical verification:

- Recompute tag: $t' \leftarrow S_{k(m)}$
- Accept if $t = t'$, reject otherwise
- Deterministic verification

MAC Security Models

Existential Unforgeability under Chosen Message Attack

Security goal: Adversary cannot forge valid tags for new messages

Attack model:

- Adversary can request tags for chosen messages (oracle access)
- Adversary wins if it produces valid tag for unqueried message
- Models realistic scenario where adversary sees many message-tag pairs

Why chosen message attacks:

- Adversary may influence some communications
- Protocol messages have predictable structure
- Interactive systems respond to adversary inputs

Standard notion: EUF-CMA (Existential Unforgeability under Chosen Message Attack)

MAC Forgeability Game

MAC Forgeability Game

$k \xleftarrow{\$} \mathcal{K}$

$(m, t) \xleftarrow{\$} \mathcal{A}^{\mathcal{O}}()$

Return $V_{k(m,t)} \wedge m \notin Q$

Oracle $\mathcal{O}(m)$

$Q \leftarrow Q \cup \{m\}$

Return $S_{k(m)}$

Adversary wins if:

- Tag t is valid for message m under key k
- Message m was never queried to oracle $\mathcal{O}$

MAC advantage: $\mathrm{Adv}_{\mathcal{J}}^{\mathrm{MAC}}(\mathcal{A}) \coloneqq \Pr[\mathcal{A} \text{ wins}]$

Security: MAC is secure if advantage is negligible for all efficient adversaries

Replay Attacks and Limitations

Replay attack scenario:

1. Alice sends authenticated message: “Transfer £1,000 to Bob” with valid tag
2. Bank verifies tag and processes transfer
3. Attack: Adversary resends same (m, t) multiple times
4. Bank processes each duplicate as legitimate transfer

Why MACs don’t prevent replay:

- Verification is stateless
- Same message always produces same tag
- No built-in freshness mechanism

Common defenses:

- Sequence numbers: Include monotonic counter
- Timestamps: Include current time in message
- Nonces: Use random values for each message
- Session state: Maintain connection-specific context

Important: Replay protection requires higher-level protocol design

Timing Attack on Verification

Vulnerable verification:

```
function verify(k, m, t):
    t' = compute_tag(k, m)
    return t == t'  // Byte-by-byte comparison
```

Attack principle:

- Comparison stops at first differing byte
- Timing reveals position of first wrong byte
- Try all 256 values for each byte position
- Complexity: $256 \times \text{tag length}$ instead of $256^{\text{tag length}}$

Real example: Xbox 360 vulnerability

- 2.2ms per byte comparison time
- Attackers forged tags for pirated games
- Exploited timing differences in MAC verification

Mitigation: Constant-time comparison

```
function secure_verify(k, m, t):
    t' = compute_tag(k, m)
    return constant_time_equals(t, t')
```

Constructing MACs from PRFs

Simple MAC from PRF (Fixed Length)

Construction: Let $F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ be a PRF

For fixed-length messages (n-bit messages):

- Sign: $S_k(m) \coloneqq F(k, m)$
- Verify: $V_k(m, t) \coloneqq 1$ if $t = F(k, m)$, else 0

Theorem: If F is a secure PRF, then the above construction gives a secure MAC for fixed-length messages.

For every q-query adversary $\mathcal{A}$, there exists adversary $\mathcal{B}$ such that:

$$\mathrm{Adv}_{\mathcal{I}}^{\mathrm{MAC}}(\mathcal{A}) \leq \mathrm{Adv}_{F}^{\mathrm{PRF}}(\mathcal{B}) + \frac{1}{|\mathcal{Y}|}$$

Limitation: Only handles messages of exactly n bits

Real-world need: Variable-length messages

Naive Extension to Variable Length

Naive approach: Split message into blocks, tag each separately

Construction:

- Parse m into blocks $m_1, m_2, ..., m_\ell$ (each n bits)
- Compute $t_i \leftarrow F(k, m_i)$ for each block i
- Tag: $t = t_1 \parallel t_2 \parallel ... \parallel t_\ell$

Security analysis: Each block independently secure

But multiple attacks possible:

1. Block reordering: $(m_1, m_2) \rightarrow (m_2, m_1)$ with (t_2, t_1)
2. Truncation: Drop last blocks and corresponding tags
3. Block substitution: Replace blocks from different authenticated messages

Conclusion: Naive block-wise tagging is insecure

Secure Variable-Length MAC Construction

Defenses against block attacks:

1. Message ID: Include random r with each block (prevents mixing)
2. Block index: Include position i with each block (prevents reordering)
3. Message length: Include length ℓ with each block (prevents truncation)

Secure construction:

Sign(k, m):

- Choose random message ID: $r \xleftarrow{\$} \{0,1\}^{\frac{n}{4}}$
- Parse m into blocks $m_1, ..., m_\ell$ (pad final block)
- For each block i: $t_i \leftarrow F(k, r \parallel \ell \parallel i \parallel m_i)$
- Return $t = \langle r, t_1, ..., t_\ell \rangle$

Verify(k, m, t):

- Parse $t = \langle r, t_1, ..., t_{\{\ell'\}} \rangle$ and m into blocks
- Check $\ell' = \ell$ and $V_{k(r \parallel \ell \parallel i \parallel m_i, t_i)} = 1$ for all i

Each block tag includes: Message ID + Length + Position + Content

Security Analysis of Enhanced Construction

Theorem: If underlying fixed-length MAC is secure, then the enhanced construction is secure for variable-length messages.

Proof structure: Consider three cases for adversary forgery

Case 1: Message ID collision (r reused)

- Probability: $\frac{q^2}{2^{\frac{n}{4}}}$ (birthday bound)
- Negligible for appropriate parameters

Case 2: New block introduced (not previously authenticated)

- Would break underlying MAC security
- Contradicts MAC security assumption

Case 3: No collision, no new blocks

- All blocks previously authenticated with same r, ℓ, i values
- But this means exact same message → not a forgery
- Probability: 0

Combining cases: Total forgery probability is negligible

Efficiency Issues with Block-by-Block MAC

Performance problems:

Computation: For message of length $d \times n$ bits:

- Need $4d$ block cipher evaluations
- Each block needs $r \parallel \ell \parallel i \parallel m_i$ formatting
- Much slower than CBC-like chaining

Tag size: Tag length is $> 4dn$ bits

- Each block produces separate tag
- Massive overhead for long messages
- Network and storage inefficient

Comparison for 1024-bit message (AES blocks):

- Block-by-block: 64 AES calls, 8192-bit tag
- CBC-MAC: 8 AES calls, 128-bit tag

Need: More efficient constructions while maintaining security

CBC-MAC: Efficient MAC Construction

CBC-MAC Construction

Idea: Chain blocks together like CBC encryption, output only final block

Let $F : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ be a PRF (block cipher)

Sign(k, m) for message of length ℓn bits:

1. Parse m into ℓ blocks: $m_1, m_2, ..., m_\ell$
2. Set $t_0 \leftarrow 0^n$ (initialization vector)
3. For $i = 1, ..., \ell$: $t_i \leftarrow F(k, t_{i-1} \oplus m_i)$
4. Return tag t_ℓ

Verify(k, m, t):

- Check message length is multiple of n
- Return 1 if $t = \text{Sign}(k, m)$, else 0

Efficiency: Only ℓ PRF evaluations, n-bit tag (optimal!)

CBC-MAC vs CBC Encryption

Similarities:

- Both use chaining structure: $t_i = F(k, t_{i-1} \oplus m_i)$
- Sequential processing of message blocks
- Based on block cipher primitive

Critical differences:

Aspect	CBC Encryption	CBC-MAC
IV	Random for each message	Fixed: 0^n
Output	All intermediate values t_i	Only final value t_ℓ
Security goal	CPA security	Unforgeability
Randomization	Required (via IV)	Deterministic

Warning: Using random IV in CBC-MAC breaks security!

Why Fixed IV in CBC-MAC?

Attack on CBC-MAC with random IV:

Given CBC-MAC with random IV r, adversary can:

1. Query message m to get tag (r, t) where $t = F(k, r \oplus m)$
2. For any target message m^*, choose $r^* = r \oplus m \oplus m^*$
3. Then tag (r^*, t) is valid for message m^* because: $F(k, r^* \oplus m^*) = F(k, (r \oplus m \oplus m^*) \oplus m^*) = F(k, r \oplus m) = t$

Why fixed IV works:

- No adversary control over initial state
- Deterministic computation for same message
- Security relies on PRF properties of block cipher

Principle: MAC security often requires deterministic behavior

Why Only Output Final Block?

Attack on CBC-MAC outputting all intermediate values:

If CBC-MAC outputs $(t_1, t_2, ..., t_\ell)$ instead of just t_ℓ:

1. Query 2-block message (m_1, m_2) to get tag (t_1, t_2) where $t_1 = F(k, m_1)$ and $t_2 = F(k, t_1 \oplus m_2)$
2. Forgery: For any m^*, the pair $(m^*, m_2 \oplus t_1 \oplus F(k, m^*))$ has the same final tag t_2
3. This gives a valid tag for unqueried message

Why final block only is secure:

- Intermediate values reveal internal state
- Adversary can exploit knowledge of intermediate states
- Final output only provides minimal information

Lesson: MAC design requires careful output selection

Fixed-Length Message Requirement

Current CBC-MAC limitation: Only secure for fixed-length messages

Attack on variable-length CBC-MAC:

1. Query 1-block message m to get tag $t = F(k, m)$
2. Forgery: 2-block message (m, t) has tag: $F(k, F(k, m) \oplus t) = F(k, t \oplus t) = F(k, 0) =$ constant
3. Can compute this constant by querying any 2-block message (x, x)

Why this breaks security:

- Different length messages allow adversary to control intermediate states
- Can construct collisions between messages of different lengths

Solution needed: Extend CBC-MAC to handle variable-length messages securely

Extending CBC-MAC to Variable Length

Method 1: Length prefixing

- Prepend message m with its length $|m|$ (encoded as n-bit string)
- Compute CBC-MAC on $(|m| \,\|\, m)$
- Prevents length-based attacks by making length part of authentication

Method 2: Two-key approach

- Use two independent keys: k_1, k_2
- Compute $t \leftarrow \text{CBC-MAC}_{k_1}(m)$
- Output final tag: $t' \leftarrow F(k_2, t)$
- Additional encryption step prevents length extension

Why these work:

- Length prefixing: Different lengths → different first blocks
- Two-key: Final transformation isolates internal state

Standard implementations:

- NIST FIPS PUB 113: Uses length prefixing approach
- ISO/IEC 9797-1: Defines both methods with variations

Lecture Summary

Summary: Message Integrity

Key concepts covered:

1. Beyond encryption: Confidentiality ≠ Integrity
   - Active adversaries can modify messages
   - Authentication essential for message trust
2. MAC security model: EUF-CMA
   - Existential unforgeability under chosen message attack
   - Adversary gets oracle access to MAC generation
   - Must forge tag for unqueried message
3. Implementation challenges:
   - Timing attacks on verification
   - Replay attacks (protocol-level defense needed)
   - Careful design required for security

Fundamental: Message integrity requires cryptographic authentication

Summary: MAC Constructions

Construction progression:

1. Fixed-length PRF-based MAC: $S_{k(m)} = F(k, m)$
   - Secure for n-bit messages
   - Direct application of PRF security
2. Naive variable-length: Independent block tagging
   - Vulnerable to reordering, truncation, mixing
   - Requires additional structure
3. Secure variable-length: Enhanced with metadata
   - Include message ID, length, position
   - Prevents all block-based attacks
   - But inefficient (large tags, many PRF calls)

4. CBC-MAC: Efficient chaining construction
   - ℓ PRF calls for ℓ blocks
   - n-bit tag (optimal size)
   - Requires careful parameter choices (fixed IV, final output only)

Summary: CBC-MAC Design Principles

Critical design decisions:

- Fixed initialization vector: $t_0 = 0^n$ (not random)
- Final block only: Output t_ℓ only (not all t_i)
- Length restrictions: Fixed-length or explicit length handling

Extensions for variable length:

- Length prefixing: Authenticate $(|m| \,||\, m)$
- Two-key method: $F\left(k_2, \text{CBC-MAC}_{k_1}(m)\right)$

Standardization:

- NIST FIPS PUB 113
- ISO/IEC 9797-1

- Widely deployed in practice

Achievement: Practical, efficient MAC from any secure block cipher

Looking Ahead

Next topics:

- Hash functions: One-way functions and collision resistance
- HMAC: MAC construction using hash functions

Message authentication is fundamental building block for secure protocols

Questions?

Lecture 9: Message Authentication Codes (MACs)

Durham University
Department of Computer Science